

Personalized Content Discovery

Netflix Prize Recommendation System

Recommendation systems power personalization across Netflix, Amazon, and Spotify. This project develops a personalized recommendation engine for the Netflix Prize Dataset — tackling massive scale (100M+ ratings) and extreme sparsity (98%+ empty user-item matrix). Core deliverables: predict unseen ratings and generate Top-K recommendations.



NETFLIX PRIZE DATASET



Users
480,189

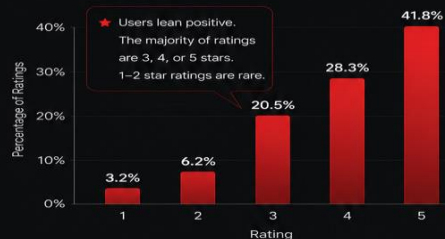


Movies
17,770



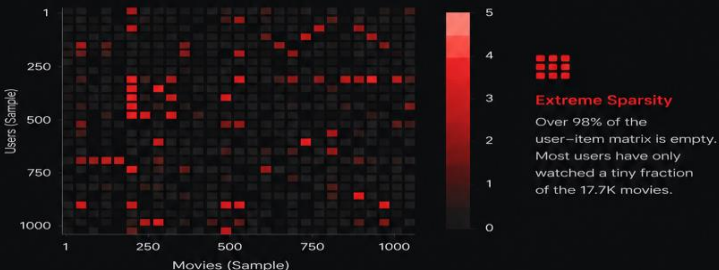
Ratings
100,480,507

RATING DISTRIBUTION

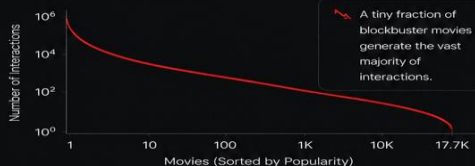


Over 98% of the user-item matrix is empty.

USER-ITEM INTERACTION HEATMAP



MOVIE POPULARITY (LONG TAIL)



Popularity Bias (Long Tail)

A tiny fraction of blockbuster movies generate the vast majority of interactions.

Key Insights from Exploratory Data Analysis

Skewed Ratings

Users lean positive. The majority of ratings are 3, 4, or 5 stars. 1-2 star ratings are rare.

Extreme Sparsity

Over 98% of the user-item matrix is empty. Most users have only watched a tiny fraction of the 17.7K movies.

Popularity Bias (Long Tail)

A tiny fraction of blockbuster movies generate the vast majority of interactions.

Business Implication

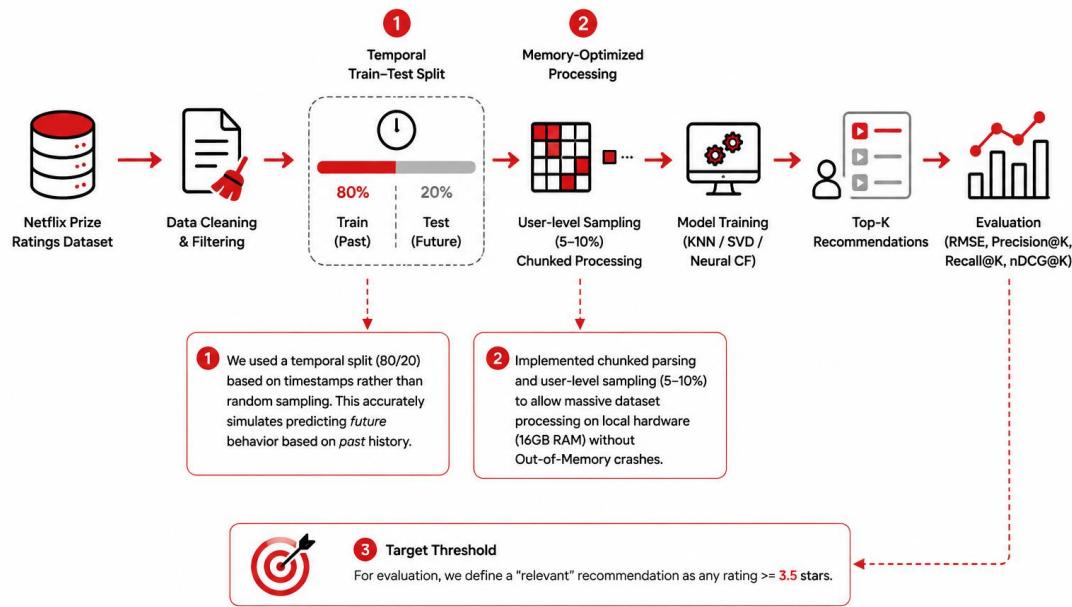
Simple Memory-based Collaborative Filtering will struggle to find robust overlapping neighbors. We need models that can fill in the "blanks."



Business Implication

Simple Memory-based Collaborative Filtering will struggle to find robust overlapping neighbors. We need models that can fill in the "blanks."

Data Pipeline & Methodology



1 Temporal Train-Test Split

We used a temporal split (80/20) based on timestamps rather than random sampling. This accurately simulates predicting *future* behavior based on *past* history.

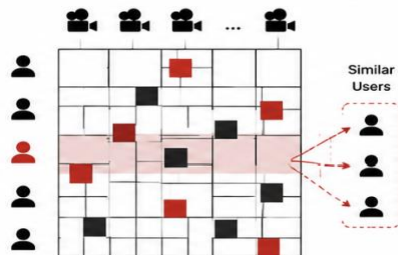
2 Memory-Optimized Processing

Implemented chunked parsing and user-level sampling (5-10%) to allow massive dataset processing on local hardware (16GB RAM) without Out-of-Memory crashes.

3 Target Thresholds

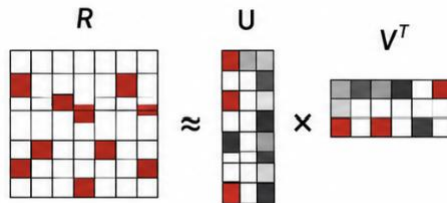
For evaluation, we define a "relevant" recommendation as any rating ≥ 3.5 stars.

Model Architectures



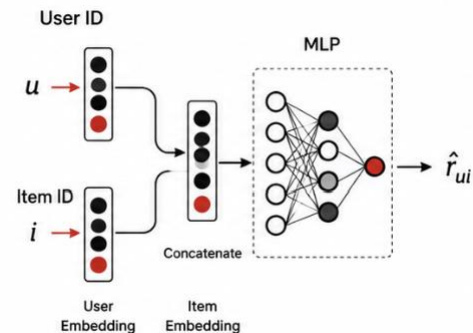
User/Item KNN (Memory-Based)

Finds users with similar tastes and recommends what they liked. Highly interpretable, but struggles heavily with sparsity and scales poorly.



SVD (Matrix Factorization)

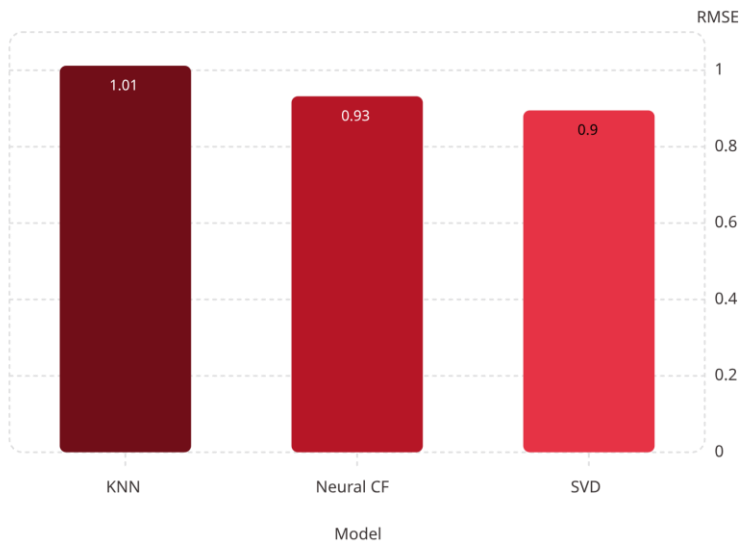
Maps users and items to a shared 100-dimensional latent space. Handles sparsity beautifully, fast inference, strong baseline.



Neural CF (Deep Learning)

Combines Generalized Matrix Factorization (GMF) with a Multi-Layer Perceptron (MLP). Captures non-linear complex patterns, GPU accelerated, but requires strict regularization.

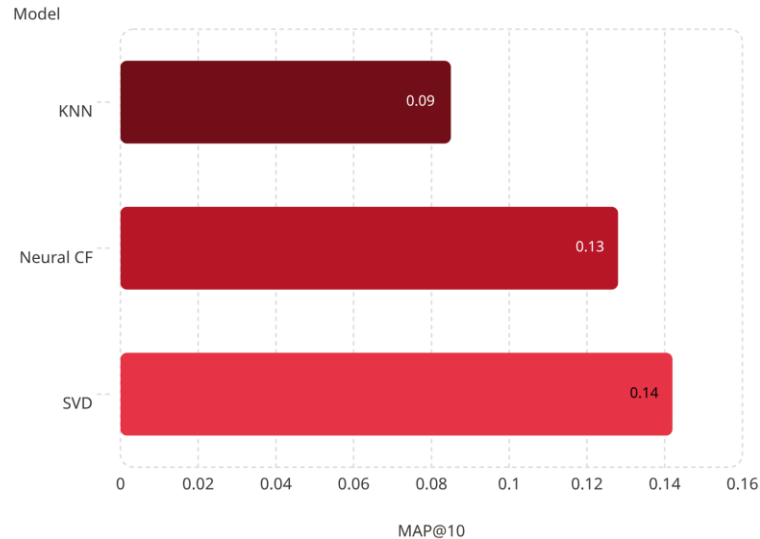
Evaluation & Metrics



RMSE

Measures how close our predicted 1–5 star rating is to reality. **SVD wins** with 0.895.

MAP@10 Ranking Quality



MAP@10

Measures how well the Top-10 recommendations align with what the user *actually* watched and liked (≥ 3.5) in the future. **SVD provides the most relevant Top-10 rankings** at 0.142.

Recommendation Quality: Success & Failures

Success Case — Mainstream User

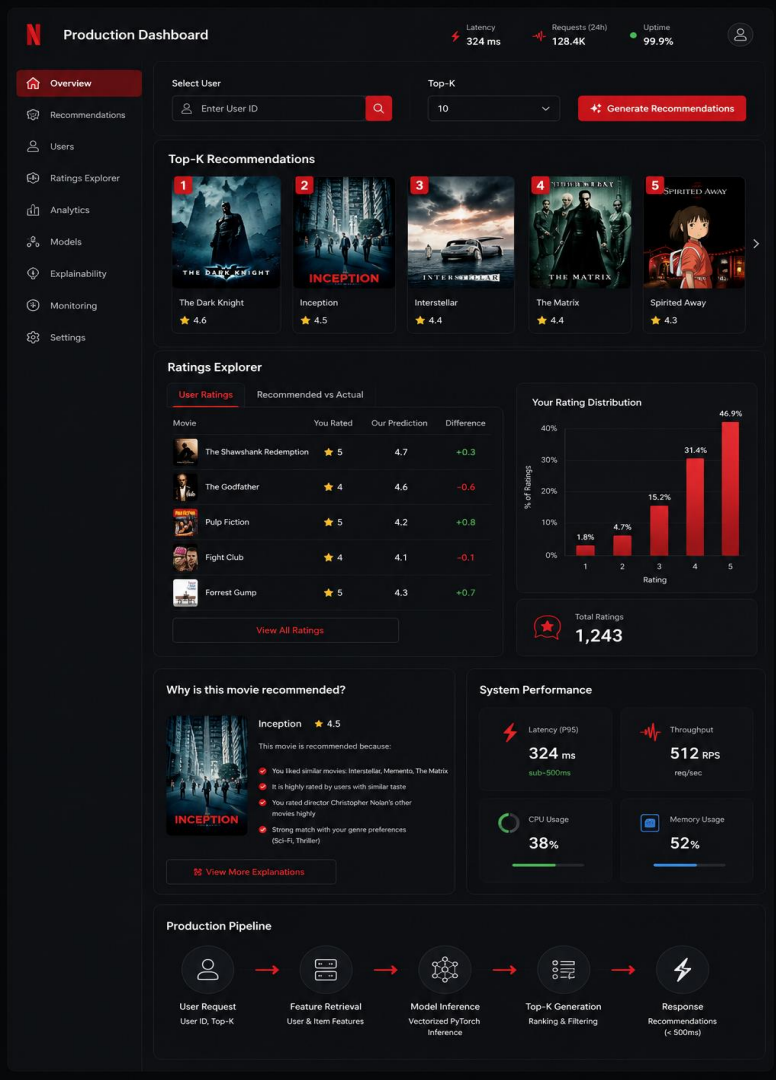
For a user who loved *The Matrix* and *Terminator 2*, the model accurately recommends *Aliens* and *Blade Runner*.

Failure Case — Cold Start

Users with <5 ratings receive generic blockbusters. SVD falls back to item popularity biases.

Failure Case — Niche Tastes

Occasionally, the model forces highly popular titles (*Lord of the Rings*) onto users with strictly niche indie tastes.



Production System Dashboard



Interactive Deployment via Streamlit

Features real-time Top-K generation for arbitrary users.



Historical Ratings Explorer

Allows exploring historical ratings side-by-side with recommendations.



Explainability Module

Includes an "Explainability" module showing *why* a movie was recommended.



Sub-500ms Latency

Achieves sub-500ms latency powered by vectorized PyTorch mini-batches.

Conclusion & Future Work

🏆 **Winner:** SVD Matrix Factorization provides the best balance of speed, accuracy, and ranking quality for the highly sparse Netflix dataset.



Next Step 1: Hybridization

Inject metadata (genres, cast) to solve the cold-start problem.



Next Step 2: Diversity Penalty

Rerank final predictions to penalize over-recommended blockbusters.



Next Step 3: Ranking Loss Tuning

Train Neural CF directly on ranking metrics (BPR loss) rather than regression (MSE).